

CHƯƠNG 7. NGUYÊN TẮC THIẾT KẾ

Áp dụng cho hệ thống Pet Service

Nhóm 23 – Trần Đức Nam Anh, Nguyễn Đức Hiếu, Triệu Trường Giang

Chương này đánh giá cách tổ chức mã nguồn của hệ thống Pet Service dựa trên hai khái niệm Coupling, Cohesion và năm nguyên lý SOLID. Nội dung được đối chiếu với cấu trúc frontend React/Vite và backend Express sau quá trình tái cấu trúc theo module. Qua đó, nhóm tập trung làm rõ những điểm thiết kế đã được áp dụng hiệu quả, đồng thời nêu một số hướng hoàn thiện có thể triển khai dần mà không làm thay đổi toàn bộ hệ thống.

7.1. Áp dụng Design Concepts

7.1.1. Giới thiệu chung về Coupling và Cohesion

Cohesion thể hiện mức độ liên quan giữa các thành phần nằm trong cùng một file hoặc module. Một module có cohesion cao thường tập trung vào một nhóm trách nhiệm cụ thể, chẳng hạn quản lý thú cưng, lịch hẹn hoặc lưu trữ. Khi cách phân chia này rõ ràng, lập trình viên có thể tìm đúng vị trí cần sửa, giảm nguy cơ tác động đến những chức năng không liên quan.

Coupling là mức độ phụ thuộc giữa các file hoặc module. Coupling thấp không có nghĩa là các phần tách biệt hoàn toàn, mà là chúng trao đổi với nhau thông qua những đầu mối và contract dễ hiểu. Trong Pet Service, component giao diện gọi service, service frontend gọi Express API và backend sử dụng middleware cùng các module nghiệp vụ. Cách tổ chức theo lớp như vậy giúp thay đổi ở một tầng ít lan sang các tầng còn lại.

Hai khái niệm này bổ sung cho nhau: mỗi module nên có trách nhiệm đủ tập trung, nhưng quan hệ giữa các module cũng cần được kiểm soát. Đây là tiêu chí phù hợp để xem xét cả cấu trúc hiện tại lẫn kết quả tái cấu trúc của dự án.

7.1.2. Đánh giá phía Frontend

7.1.2.1. Cohesion phía Frontend

Những điểm đã làm tốt:

- Frontend được chia theo các nhóm người dùng gồm admin, customer, doctor, staff và auth. Mỗi nhóm có page, component và service tương ứng nên phạm vi chức năng khá rõ, thuận tiện khi cần phát triển hoặc kiểm tra một luồng nghiệp vụ.
- Bên trong từng nhóm, component tiếp tục được chia theo nghiệp vụ. Phần customer có các khu vực appointments, boarding, history, medications, notifications, pets và profile.

Cấu trúc này bám sát chức năng đã mô tả trong tài liệu ITSSfinal2, vì vậy mối liên hệ giữa yêu cầu và mã nguồn dễ theo dõi.

- Các thao tác gọi API của khách hàng đã được tách thành những file như `customerAppointmentsApi.ts`, `customerBoardingApi.ts` và `customerPetsApi.ts`. Component chủ yếu tập trung vào trạng thái và hiển thị, không cần biết chi tiết bảng hoặc câu truy vấn ở backend.
- Thư mục `components/ui` cung cấp Button, Card, Dialog, Table, Input và các thành phần dùng chung. Việc tái sử dụng này vừa tạo giao diện thống nhất, vừa giảm số đoạn JSX và xử lý tương tác bị lặp lại giữa các màn hình.
- TypeScript được sử dụng để mô tả dữ liệu của thú cưng, lịch hẹn, thuốc và thông báo. Kiểu dữ liệu rõ ràng giúp component nhận đúng trường cần thiết và hỗ trợ phát hiện sai lệch ngay trong quá trình lập trình.

Định hướng hoàn thiện thêm:

- Một số màn hình lớn như CustomerPortal hoặc StaffBoardingTab có thể tiếp tục tách thành component hiển thị, modal và custom hook. Đây là bước tinh gọn tiếp theo để mỗi file tập trung hơn, không phải dấu hiệu cấu trúc frontend đang sai.
- Có thể thống nhất toàn bộ thao tác HTTP qua một helper dùng chung cho customer, doctor, staff và admin. Khi đó việc gắn token, parse JSON và xử lý lỗi sẽ đồng nhất hơn trên toàn frontend.

7.1.2.2. Coupling phía Frontend

Những điểm đã làm tốt:

- Frontend không truy cập Supabase trực tiếp mà trao đổi với Express API. Vì vậy giao diện không bị gắn chặt với tên bảng, tên cột hoặc cơ chế xác thực của cơ sở dữ liệu.
- Các helper `apiUrl.ts`, `authSession.ts` và `requestJson.ts` tạo thành những đầu mối dùng chung cho địa chỉ backend, phiên đăng nhập và request JSON. Cách tổ chức này đã giảm đáng kể code lặp và giúp các service có hình thức tương đối thống nhất.
- Các interface TypeScript đóng vai trò contract giữa service và component. Khi một component nhận danh sách lịch hẹn hoặc hồ sơ thú cưng, cấu trúc dữ liệu mong đợi đã được mô tả rõ thay vì phụ thuộc vào suy đoán.
- Nhiều component giao tiếp thông qua props nhỏ và callback cụ thể. Điều này làm giảm việc truy cập trực tiếp vào trạng thái của component cha, đồng thời giúp component con dễ tái sử dụng hơn.
- Các file tương thích và re-export giữ cho đường dẫn import cũ vẫn hoạt động trong thời gian tái cấu trúc. Đây là lựa chọn thực tế giúp nhóm thay đổi kiến trúc từng bước mà không buộc toàn bộ frontend phải sửa cùng lúc.

Định hướng hoàn thiện thêm:

- Backend nên ưu tiên trả về mã trạng thái và nhãn nghiệp vụ; màu sắc hoặc class Tailwind có thể ánh xạ ở frontend để ranh giới trình bày rõ hơn.

- Một cấu hình ánh xạ role với portal và một chuẩn response chung như { ok, data, message } sẽ giúp bổ sung vai trò hoặc endpoint mới thuận lợi hơn.

7.1.3. Đánh giá phía Backend

7.1.3.1. Cohesion phía Backend

Những điểm đã làm tốt sau tái cấu trúc:

- app.js và server.js đã được tách theo hai trách nhiệm khác nhau. app.js tạo Express app, thiết lập middleware và gắn route; server.js phụ trách khởi động tiến trình cùng lịch gửi email nhắc nhở. Đây là thay đổi rõ ràng theo hướng cohesion cao hơn.
- Backend được sắp xếp theo các module nghiệp vụ như auth, customers, pets, appointments, staff, doctors, medical, boarding, billing và notifications. Mỗi nhóm chức năng có vị trí dễ nhận biết, phù hợp với miền nghiệp vụ của hệ thống chăm sóc thú cưng.
- Các phân dùng chung đã được đưa về config, middlewares, lib và utils. Middleware xác thực và kiểm tra role được tái sử dụng cho nhiều nhóm người dùng, nhờ đó route không phải lặp lại logic bảo mật.
- modules/index.js đóng vai trò danh sách đăng ký module và prefix API. app.js chỉ duyệt danh sách này để gắn route, nên phần khởi tạo ứng dụng gọn và ít phụ thuộc vào chi tiết của từng endpoint.
- Nhiều service đã được chuyên biệt theo chức năng customer, doctor, boarding, notification hoặc billing. Route có thể gọi service thay vì tự xử lý toàn bộ nghiệp vụ, giúp cấu trúc backend tiến gần hơn đến mô hình route mỏng.
- Quá trình tái cấu trúc vẫn giữ các điểm tương thích cần thiết với luồng hiện tại. Điều này giúp kiến trúc được cải thiện nhưng hạn chế rủi ro làm gián đoạn các chức năng đã hoạt động.

Định hướng hoàn thiện thêm:

- medical.routes.js và một số service lớn của admin, staff có thể được chia dần theo use case như khám bệnh, đơn thuốc, báo cáo, grooming, boarding và thanh toán. Nên ưu tiên các phần thay đổi thường xuyên trước.
- Logic của reports và grooming có thể chuyển hoàn toàn về module sở hữu nghiệp vụ thay vì chỉ tái xuất hàm. Bước này sẽ làm ranh giới module rõ hơn nhưng không yêu cầu viết lại các API hiện có.

7.1.3.2. Coupling phía Backend

Những điểm đã làm tốt:

- Các route được đăng ký thông qua apiModules, vì vậy app.js không cần biết chi tiết endpoint của từng miền nghiệp vụ. Việc thêm một nhóm route mới chủ yếu diễn ra tại module và danh sách đăng ký.
- JWT, Prisma client và Supabase client được đặt tại các thư viện dùng chung. Các service tái sử dụng cùng một đầu mối thay vì tự tạo kết nối hoặc tự cài đặt lại cách xác thực.

- Middleware xác thực và phân quyền dùng chung cho customer, doctor, staff và admin. Chính sách truy cập vì vậy được giữ ở một lớp rõ ràng, hạn chế việc mỗi handler diễn giải quyền theo một cách khác nhau.
- Ranh giới route – service đã xuất hiện ở phần lớn luồng nghiệp vụ. Route tiếp nhận request và trả response, trong khi service xử lý phần công việc chính. Đây là nền tảng tốt để tiếp tục giảm phụ thuộc vào framework và dữ liệu.
- Cấu hình chung đã có vị trí riêng, cho thấy dự án đã chủ động tách thông tin môi trường khỏi nghiệp vụ. Cách làm này cũng hỗ trợ triển khai trên nhiều môi trường mà không sửa trực tiếp mã nguồn.

Định hướng hoàn thiện thêm:

- Với các miền quan trọng như appointment, pet, boarding và invoice, nhóm có thể bổ sung repository để service phụ thuộc vào contract dữ liệu thay vì chi tiết Supabase. Đây là lớp mở rộng hợp lý khi nhu cầu test hoặc thay đổi nguồn dữ liệu tăng lên.
- Các tác vụ email, notification, PDF và storage có thể đi qua adapter dùng chung; toàn bộ biến môi trường cũng nên được kiểm tra tập trung khi server khởi động.

7.1.4. Kết luận

Xét theo Coupling và Cohesion, Pet Service đã có nền tảng thiết kế khá tốt. Frontend được phân chia theo vai trò và tính năng; backend sau tái cấu trúc đã tách app khỏi server, hình thành các module nghiệp vụ và gom nhiều thành phần dùng chung. Những điểm cần làm tiếp chủ yếu là tinh gọn một số file lớn và bổ sung abstraction ở nơi thật sự có nhu cầu. Vì vậy, hướng phù hợp là tiếp tục cải thiện từng phần trên cấu trúc hiện có, thay vì thay đổi toàn bộ kiến trúc.

7.2. Áp dụng Design Principles SOLID

7.2.1. Giới thiệu chung

SOLID gồm năm nguyên lý giúp mã nguồn dễ đọc, dễ sửa, dễ kiểm thử và thuận lợi khi mở rộng. Dù Pet Service không xây dựng nhiều hệ thống class kế thừa theo cách hướng đối tượng truyền thống, SOLID vẫn có thể áp dụng trực tiếp cho component React, service, module, middleware và các kiểu dữ liệu TypeScript.

- S – Single Responsibility Principle: một file hoặc module nên tập trung vào một trách nhiệm chính.
- O – Open/Closed Principle: cấu trúc nên cho phép bổ sung chức năng mới mà hạn chế sửa phần ổn định.
- L – Liskov Substitution Principle: các implementation cùng contract phải có thể thay thế nhau mà caller vẫn hoạt động đúng.

- I – Interface Segregation Principle: caller chỉ nên phụ thuộc vào interface hoặc service phù hợp với nhu cầu của nó.
- D – Dependency Inversion Principle: logic nghiệp vụ nên phụ thuộc vào abstraction thay vì chi tiết database, email hoặc storage.

7.2.2. Đánh giá việc áp dụng SOLID

7.2.2.1. Single Responsibility Principle (SRP)

Mức độ áp dụng trong dự án:

- app.js và server.js có nhiệm vụ riêng, thể hiện SRP rõ nhất sau quá trình tái cấu trúc backend.
- Các module auth, pets, appointments, boarding, billing và notifications được tổ chức theo nghiệp vụ; frontend cũng có service, type và component riêng cho nhiều chức năng customer.
- Middleware xác thực, cấu hình môi trường và client dữ liệu đã có vị trí dùng chung, tránh để từng route cùng gánh thêm các trách nhiệm này.
- Các component UI cơ sở tập trung vào hiển thị và tương tác chung, trong khi page và portal đảm nhiệm việc ghép thành luồng nghiệp vụ hoàn chỉnh.

Bước hoàn thiện phù hợp:

- Tiếp tục tách các file lớn theo use case và custom hook để mỗi file có ít lý do thay đổi hơn; ưu tiên medical, admin, staff và các portal chứa nhiều trạng thái.

7.2.2.2. Open/Closed Principle (OCP)

Mức độ áp dụng trong dự án:

- Danh sách apiModules cho phép thêm một nhóm route bằng cách đăng ký module mới, không cần đưa toàn bộ handler vào app.js.
- Thư viện component UI và cách chia feature giúp bổ sung page hoặc màn hình mới mà ít phải sửa các thành phần giao diện đã ổn định.
- Public API của module và các file re-export hỗ trợ thay đổi cấu trúc bên trong trong khi vẫn giữ cách gọi quen thuộc cho phần còn lại của hệ thống.
- Các kiểu TypeScript và mapping nghiệp vụ tạo vị trí tập trung để mở rộng dữ liệu hoặc trạng thái, thay vì phân tán logic trên nhiều component.

Bước hoàn thiện phù hợp:

- Bổ sung registry cho role, portal và handler trạng thái để giảm số câu lệnh if hoặc switch cần sửa khi hệ thống xuất hiện vai trò và quy trình mới.

7.2.2.3. Liskov Substitution Principle (LSP)

Trong Pet Service, LSP phù hợp hơn khi xét khả năng thay thế các implementation cùng contract, thay vì quan hệ class cha – class con. Dự án đã có TypeScript interface ở

frontend và các response/DTO tương đối rõ ở nhiều luồng. Đây là nền tảng để caller sử dụng dữ liệu theo một cấu trúc ổn định mà không cần biết cách dữ liệu được lấy từ đâu.

Những nền tảng đã có:

- Component dựa vào kiểu dữ liệu của service, không dựa trực tiếp vào cấu trúc bảng Supabase.
- Các module backend trao đổi qua hàm và dữ liệu trả về có mục đích rõ, giúp việc kiểm tra contract thuận lợi hơn.
- Việc gom Prisma và Supabase client về thư viện dùng chung tạo một điểm kiểm soát trước khi phát triển repository có thể thay thế.

Bước hoàn thiện phù hợp:

- Định nghĩa repository contract cho một số miền chính và viết test dùng chung cho implementation thật, implementation giả. Khi đó khả năng thay Supabase bằng Prisma hoặc mock trong unit test sẽ được thể hiện rõ hơn.

7.2.2.4. Interface Segregation Principle (ISP)

Mức độ áp dụng trong dự án:

- API phía customer được chia theo appointment, boarding, medication, notification và pet. Mỗi màn hình chỉ import nhóm hàm phục vụ đúng nghiệp vụ của nó.
- Nhiều component nhận những props cụ thể thay vì toàn bộ đối tượng portal, nhờ đó phạm vi phụ thuộc nhỏ và dễ tái sử dụng.
- Backend phân tách route theo module; middleware cũng có vai trò riêng cho xác thực, phân quyền và xử lý request.
- TypeScript interface giúp mô tả chính xác dữ liệu cần dùng, tránh buộc component phải biết những trường không liên quan đến giao diện của nó.

Bước hoàn thiện phù hợp:

- Tách thêm các service doctor, staff và admin theo nhóm BoardingApi, PaymentApi, DoctorExamApi hoặc NotificationApi; đồng thời chỉ export những hàm thực sự là public API của module.

7.2.2.5. Dependency Inversion Principle (DIP)

Mức độ áp dụng trong dự án:

- Component frontend phụ thuộc vào service thay vì cơ sở dữ liệu. Đây là sự đảo chiều phụ thuộc quan trọng, giúp UI chỉ quan tâm đến nhu cầu nghiệp vụ.
- Route backend phần lớn chuyển công việc cho service, trong khi xác thực và phân quyền được giao cho middleware dùng chung.
- Supabase client, Prisma client và JWT helper được gom trong lib, hạn chế việc khởi tạo chi tiết hạ tầng rườm rà.
- Cấu trúc config, module và utility sau tái cấu trúc tạo nền tảng để truyền dependency và thay thế implementation trong tương lai.

Bước hoàn thiện phù hợp:

- Bổ sung các abstraction đơn giản như AppointmentRepository, EmailSender, FileStorage và PdfGenerator ở những luồng cần test độc lập. Implementation thật được cấu hình khi khởi động, còn implementation giả dùng cho unit test.

7.2.3. Tổng hợp và hướng phát triển

Nguyên lý	Mức độ áp dụng	Nhận xét trọng tâm
SRP	Tốt	Cấu trúc module rõ; app, server, middleware và nhiều service đã có trách nhiệm riêng.
OCP	Khá	Cơ chế đăng ký module và component dùng chung hỗ trợ mở rộng tương đối thuận lợi.
LSP	Khá – đang hoàn thiện	TypeScript và DTO tạo nền tảng contract; repository sẽ làm khả năng thay thế rõ hơn.
ISP	Khá	API phía customer đã tách nhỏ; các nhóm doctor, staff và admin có thể tiếp tục chia theo use case.
DIP	Khá – đang hoàn thiện	Service layer và thư viện dùng chung đã có; abstraction cho dữ liệu và dịch vụ ngoài là bước tiếp theo.

Kết quả tổng hợp cho thấy các nguyên lý SOLID đã xuất hiện ở nhiều quyết định kiến trúc quan trọng của dự án. SRP và ISP thể hiện rõ qua cách chia module, service và component; OCP được hỗ trợ bởi cơ chế đăng ký module và thành phần dùng chung. LSP và DIP chưa cần triển khai theo mô hình phức tạp ngay từ đầu, nhưng dự án đã có đủ nền tảng để bổ sung repository hoặc adapter khi quy mô kiểm thử và tích hợp tăng lên.

Các công việc nên ưu tiên trong giai đoạn tiếp theo:

- Tách medical.routes.js và các service lớn theo use case, thực hiện từng phần để dễ kiểm tra hồi quy.
- Thống nhất HTTP client cho frontend và chuẩn response cho backend.
- Tách component dài thành hook, phần hiển thị và modal chuyên biệt.
- Thử nghiệm repository trước với appointment hoặc boarding, sau đó mới nhân rộng nếu mang lại lợi ích rõ ràng.
- Giữ status nghiệp vụ ở backend và ánh xạ màu sắc, class hiển thị ở frontend.

7.2.4. Kết luận

Pet Service đã áp dụng khá rõ tinh thần của thiết kế tốt và SOLID, đặc biệt qua việc tách frontend – backend, chia code theo vai trò và nghiệp vụ, sử dụng service layer, component dùng chung, middleware và cấu trúc module backend. Phân tái cấu trúc hiện tại là một bước tiến quan trọng vì app, server, config, middleware và các module đã có vị trí riêng, trong khi những luồng đang hoạt động vẫn được giữ ổn định.

Nhìn theo tỉ lệ tổng thể, phần thiết kế đã làm tốt chiếm ưu thế; các nội dung còn lại chủ yếu là cơ hội tinh gọn và tăng khả năng kiểm thử. Nhóm có thể tiếp tục cải thiện theo từng use case, ưu tiên nơi thường xuyên thay đổi hoặc có nhiều phụ thuộc. Cách phát triển từng bước này phù hợp với quy mô dự án và giúp hệ thống dễ bảo trì, mở rộng hơn mà không cần viết lại từ đầu.